The Journal of Supercomputing
https://doi.org/10.1007/s11227-021-03779-4

GPU-aware resource management in heterogeneous cloud data centers

Ashwin Kumar Kulkarni[1] · **B. Annappa**[1]

Accepted: 25 March 2021
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

Abstract

The power of rapid scalability and easy maintainability of cloud services is driving many high-performance computing applications from company server racks into cloud data centers. With the evolution of Graphics Processing Units, composing of an extensive array of parallel computing single-instruction-multiple-data processors are being considered as a platform for high-performance computing because of their high throughput. Many cloud providers have begun offering GPU-enabled services for the users where GPUs are essential (for high computational power) to meet the desired Quality-of-service. Virtual machine placement and load balancing the GPUs in the virtualized environments like the cloud is still an evolving area of research and it is of prime importance to achieve higher resource efficiency and also to save energy. The current VM placement techniques do not consider the impact of VM workload type and GPU memory status on the VM placement decisions. This paper discusses the current issues with the First Fit policy of virtual machine placement used in VMWare Horizon and proposes a GPU-aware VM placement technique for GPU-enabled virtualized environments like cloud data centers. The experiments conducted using the synthetic workloads indicate reduction in the energy consumption, reduction in search space of physical hosts, and the makespan of the system. It also presents a summary of the current challenges for GPU resource management in virtualized environments and specific issues in developing cloud applications targeting GPUs under the virtualization layer.

Keywords Cloud computing · GPU-enabled VM placement · GPU resource management

✉ Ashwin Kumar Kulkarni
ashwin.kulkarni7@gmail.com

B. Annappa
annappa@ieee.org

[1] Department of Computer Science and Engineering, National Institute of Technology Karnataka, Surathkal, Mangalore, India

Published online: 08 April 2021

1 Introduction

Cloud computing offers computing infrastructure as a commodity to users on a pay as you go model. The cloud has helped businesses to reduce the initial investments into IT infrastructure for hosting services and focus on other important business aspects. Virtualization technology [25] used in cloud data centers has helped reduce the ownership costs by efficiently sharing the single physical server among multiple user applications running in isolation. The features such as on-demand resource scalability, fault tolerance, and service offering in three distinct models such as Infrastructure-as-a-Service (IaaS), Software-as-a-Service (SaaS), Platform-as-a-Service (PaaS) have made cloud computing [17] a lucrative option to consider. Cloud providers are hosting services as simple as static webpages to more complex real-time applications such as video streaming, transcoding, and business analytics solutions.

In recent years GPUs are gaining importance for their massively parallel computing capability and most of the commercial computing platforms (devices) sold in the market today have variants of the GPU installed. Though GPUs were initially used only for graphics applications like gaming, display, etc., from past few years, GPUs are regarded as a high throughput parallel computing platforms suitable for general purpose computing [13]. The applications such as machine learning, medical imaging, inference generation, support computing for smart cities and infrastructure development are leveraging GPU-enabled cloud infrastructure. The neural network (machine learning) extensions for deep learning [20] and artificial intelligence built inside GPU hardware have only added to growing popularity of cloud computing.

The growing demand for GPUs has compelled cloud providers to enable GPU processing inside data centers to support hosting complex HPCs, real-time applications, and virtual desktop applications. The GPU-enabled VM instances can accelerate user applications significantly by offloading compute-intensive part of application logic onto GPU's block of parallel threads. Today, almost all mainstream cloud providers like Amazon, Microsoft, Google have one, or many types of GPU-enabled instances to offer.

The conventional techniques of VM placements used for CPU only cloud do not hold good for GPUs because of the inherent differences in terms of architectures, control software, and distributed program/memory models [11]. These differences make GPU-enabled VM placements a more complex problem and if not addressed can cause inefficiency in resource utilization and power consumption in data centers.

The modern data center is a server farm containing heterogeneous physical machines. Some of the physical machines may have one or more GPUs fitted inside them and other physical machines may not have any GPU device fitted with them. The VM placement technique has to consider the differing physical composition of the physical machines and VM workload types (GPU or non-GPU) to place VM workloads on them. Also, the GPU device memory (GDDRAM) is a critical shared resource in parallel processing on GPU in a virtual environment.

The VM placement technique has to take into account GPU device memory availability as an important parameter for placing GPU-enabled VM placements on the physical GPUs to ensure GPU resources in the cloud are used efficiently.

In this paper, the problem of optimizing power consumption by efficient utilization of heterogeneous physical machines in a data center by considering the VM workload type, physical machine composition, and bottleneck GPU resource such as GPU device memory (GDDRAM) is investigated. A GPU-aware VM placement technique to reduce the power consumption, improve the resource utilization and reducing overall makespan of the VM placement process is presented in this paper.

The contributions of the paper are as follows;

1. A brief introduction to GPUs and their virtualization solutions used in data centers. Discussion on some of the currently used GPU resource provisioning techniques.
2. Investigation of the limitations of the First Fit (FF) policy for VM placement in GPU-enabled cloud data centers.
3. A GPU-aware VM placement strategy for GPU-enabled heterogeneous cloud data centers to overcome the limitations of FF policy.
4. Evaluation of the proposed GPU-aware technique using the synthetic workloads and analysis of results with the existing well known First Fit policy to prove the effectiveness of the proposed solution.
5. Summary of challenges in the efficient management of GPU resources in a virtualized environment and issues in utilizing the virtualized GPUs in the cloud environment.

2 Related work

The GPU computing in the cloud is an emerging trend as more and more compute-intensive, HPC, and graphics applications are hosted on the cloud data centers. Though enough research has been done on CPU virtualization, the GPU virtualization and issues with the management of GPU resources in the cloud is still a growing research area. In this section, some noted past works in GPU provisioning, GPU programming framework developments, and open source simulator developments for GPU computing in the cloud are discussed.

2.1 GPU provisioning in cloud

Menychtas et al. proposed a disengaged scheduling technique for the provisioning of GPU to vGPUs. The paper proposes disengaged timeslice with overuse control [18] that ensures fairness in the allocation and disengaged fair queueing that is helpful to limit resource idle states, but the method is probabilistic. Schedulers ensure a fair share of GPU among all application even when applications are non-cooperative and adverse to each other.

Gupta et al. proposes a GVim [9] scheme that utilizes both round-robin (RR) and Xeno credit-based scheduling (XC) techniques of the Xen hypervisor for task scheduling on GPU. RR scheduling sequentially selects a vGPU for every fixed timeslice and monitors the call buffer of the vGPU's during this period. XC uses a credit concept, which is time allocated for each vGPU. XC process vGPU's call buffer for a variable time, which is proportional to the credit amount, this ensures weighted fair sharing between guest vGPUs.

Sengupta et al. propose a Rain [22] framework responsible for load balancing GPU requests across distributed machines GPUs. The work suggests a two-level hierarchical scheduling policy. The top-level module of framework distributes the load across all GPU equipped server machines. The bottom level module is responsible for GPU device level scheduling of vGPUs.

Suzuki et al. propose a GPUvm [23] scheme that uses a BAND scheduler and solves the issue with a credit-based scheduling scheme. The proposed technique solves the miscalculation of credit when GPU idle time is included in credit amount, which may lead to inappropriate GPU share for certain vGPUs. GPUvm solves this issue by first transforming the GPU scheduler's CPU time into credit value and then subtracts the total credit value from the current vGPU. Some of the important GPU provisioning policies and the associated frameworks are mentioned in Table 1.

Farooqui et al. investigated current work-stealing algorithms and suggested them to be inefficient in CPU-GPU integrated processors. Existing algorithms are found to be unaware of the CPU and GPU characteristics, and such a situation results in degradation of performance in OpenCL like applications which are capable of running on both platforms. To overcome this issue, the authors proposed a framework named Libra [6], which first derives the device affinity scores for applications. The application is assigned to the device with the highest affinity score.

Gupta et al. proposed a new Pegasus [10] framework that addressed one of the challenges in GPU scheduling, that GPU virtualization technique has no access to impose scheduling policy, because multiplexing of GPU is integrated into the device drivers. Pegasus proposed a concept called VCPU with which GPUs are made basic scheduling entities. The Pegasus includes proportional fair share, FCFS, credit-based

Table 1 Important past work related to GPU provisioning policies

Authors, year	GPU provisioning policy	Framework proposed
Menychtas et al. 2014	Fair queueing and round robin	Disengaged scheduling
Gupta et al. 2009	Round robin and credit based	GViM
Sengupta et al. 2013	Priority based and credit based	Rain
Suzuki et al. 2014	Credit based	GPUvm
Farooqui et al. 2016	Affinity based	Libra
Gupta et al. 2011	FCFS	Pegasus
Zhang et al. 2014	SLA based	VGASA

scheme, and SLA feedback-based schedulers. The objective of Pegasus is to meet the different requirements set forth by applications using different GPU schedulers.

Zhang et al. proposed VGASA [27] including adaptive scheduling policies; these adaptive algorithms include a dynamic feedback control loop. VGASA consists of three scheduling policies; SLA-aware algorithm receives FPS information and adjusts the sleep time per frame time. Fair SLA-aware algorithm takes away GPU from fast running applications and allocates to slow running ones, and enhanced SLA-aware allows all VMs to possess the same frame rate under 100% GPU utilization.

2.2 GPU API and programming frameworks

The GPUs are accessed by applications for their computing needs typically through an API interface. The virtualization techniques such as API remoting achieve virtualization of GPUs by providing a wrapper around these GPU APIs to route GPU requests to a remote GPU. Four well-known programming frameworks/models offer GPU access through their API interfaces to applications. Each framework is provided for a set of target applications.

OpenGL [26] is a graphics acceleration library with support from the GPU device. The framework is mainly developed for video game development, image and vision processing, and visualization tasks. OpenGL provides a platform-independent interface for applications and provides support for various GPU cards from different vendors.

Direct3D [3] is a Microsoft Windows proprietary graphics API provided for rendering 3D graphics content for games and visualization applications. The Direct3D provides a low-level abstracted API by hiding different vendor's GPU cards.

CUDA [21] is a programming framework developed by NVIDIA for general purpose graphics processing unit (GPGPU) programming targeted for exploiting the computing power of NVIDIA graphics cards for solving general purpose compute-intensive problems. CUDA provides C, C++ extensions for GPU operations such as device memory allocations, GPU task (kernel) executions, event handling, data transfer, and synchronization related activities.

OpenCL [15] is a parallel processing framework that can support heterogeneous parallel processing capable platforms. Like CUDA, OpenCL has support for device-specific code launch and execution, data transfer between host and OpenCL device, and memory management operations. Unlike CUDA, the OpenCL is a cross-platform framework supporting multiple vendor's GPU cards.

Vulkan [14] offered by the Khronos group is a new generation of cross-platform graphics and compute API. It provides high-efficiency access to GPUs used in personal computers, servers, consoles, mobile phones, and other embedded devices. Vulkan is a lower level API compared to OpenGL and Direct3D and offers low overhead parallel tasking.

2.3 Simulation tools supporting GPU

Following are the popular open source simulation tools supporting GPU task processing with their features and limitations.

1. **GPGPU-Sim-** The framework of GPGPU-Sim [2] provides functional and cycle-accurate simulation modes. It consists of a GPUWatch, a power simulator for GPU, and GPGPU-Sim for power usage measurement. The speed of simulation of GPGPU-Sim needs improvement and there exists no support for evaluating cloud solutions at present.
2. **Multi2Sim-** The Multi2Sim [7] offers cycle-accurate simulations of both CPU and GPU processors in the systems. Multi2Sim provides no support for cloud at present.
3. **GPUCloudSim-** The GPUCloudSim [1] provides the GPU processing simulation in the cloud environment. The tool also provides performance and interference models. At present, tool offers no feature support for GPU-enabled VM migrations.

3 GPUs and cloud datacenters

The typical block diagram of a GPU is shown in Fig. 1a. The GPUs consist of several thousands of SIMD cores packaged into streaming multiprocessors (SMs). SMs are responsible for executing GPU tasks. Each GPU has its local memory called GDDRAM (graphics double data rate) and has two copy engines that can transfer data from GPU device memory to the main memory of the server in both directions

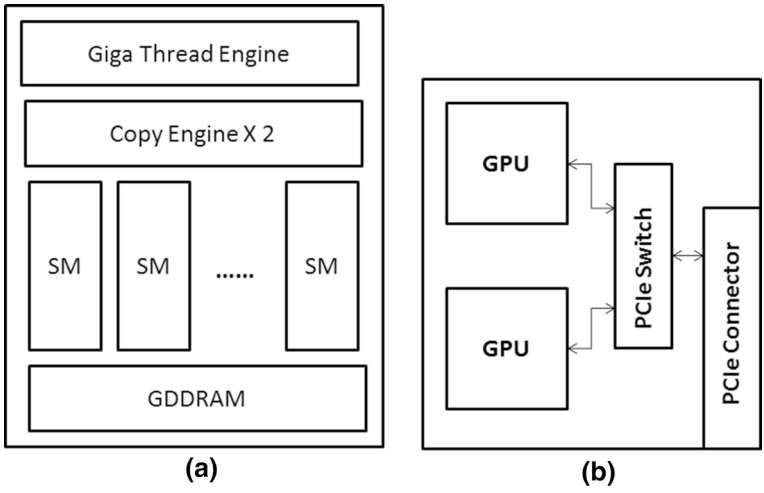


Fig. 1 Block Diagram of **a** typical GPU and **b** video card

simultaneously. The Giga Thread Engine is responsible for scheduling GPU threads onto SMs for execution. The GPU tasks are usually submitted as a group of threads called blocks to GPU for execution.

GPUs are usually fitted inside video cards, where each video card can host multiple GPUs inside it. The GPUs are interconnected inside video cards using a PCIe switch, and the video card is connected to the host machine using PCIe connector. The PCIe connector is connected to the system bus using which data flow between GPU and main memory is carried out, as shown in Fig. 1b.

The threads are distributed and scheduled inside SMs for execution. The scheduling of the thread blocks into SMs is carried out by GPU using the cooperative multitasking based on a leftover policy [1]. The steps followed in GPU task execution in an application is represented in Fig. 2.

Initially, the data required for GPU tasks are transferred from the system's main memory (RAM) to GPU device memory (GDDRAM). Once the data transfer is completed, the GPU tasks are bundled into blocks of threads and launched onto GPU for execution. After the execution of all threads is completed, results are copied back to the main memory from GPU memory.

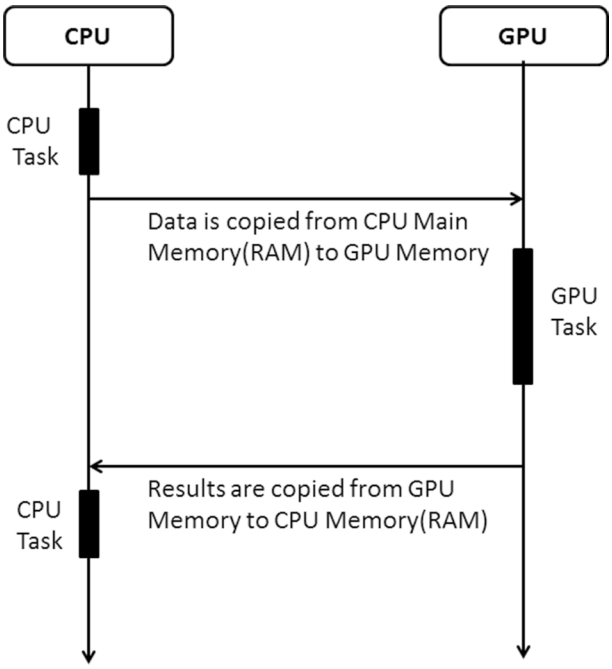


Fig. 2 Typical steps in GPU task execution in an application

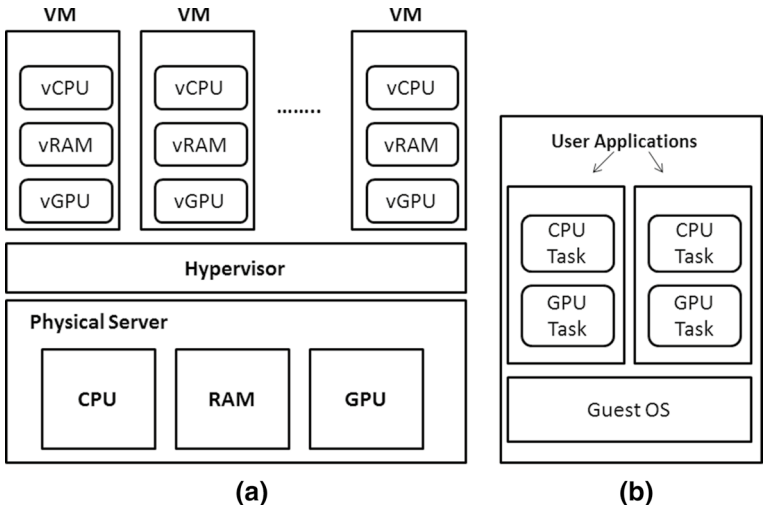


Fig. 3 **a** System view of GPU-enabled virtualized server, **b** typical VM with GPU and CPU tasks

3.1 GPU virtualization in cloud

Virtualization is employed with GPU to share the same GPU device with multiple user applications residing inside separate VMs. Virtualization helps to use resources efficiently by sharing unused computing power among different tasks.

The server scale GPUs are offered by multiple vendors such as NVIDIA, AMD, Intel, etc. Due to the unavailability of GPU virtualization support from the vendor side in earlier days, user VM was given direct pass through to GPU device (using vendor driver inside VM) for executing GPU tasks. However, in recent years, vendors have started to offer virtualization support to GPUs for using inside cloud data centers such as NVIDIA Grid technology [19]. Figure 3a shows a virtualized view of a GPU-enabled server inside the data center, and Fig. 3b depicts a typical user VM containing GPU tasks.

There are multiple virtualization methods [1] used for accessing GPU inside cloud datacenters each having their own pros and cons. We have listed some of the techniques used in brief.

1. **API remoting:** The approach virtualizes GPU at API level where calls to GPU are intercepted at the API level and are forwarded to a remote machine with GPU device for processing.
2. **Full virtualization:** The approach virtualizes GPUs at the driver level where a GPU driver is installed inside user VM to communicate with virtual GPU. It incurs a penalty in performance. The hypervisor is responsible for scheduling virtualized GPUs (vGPUs).

3. **Paravirtualization:** The approach is similar to full virtualization. However, the guest OS driver is modified to avoid performance degradation to some extent. The hypervisor is responsible for scheduling virtualized GPUs (vGPUs) in this approach too.
4. **Hardware-assisted virtualization:** The approach is supported by special hardware extensions provided by hardware vendors. These hardware extensions are responsible for VM to GPU mappings and parallel (multiplexing) executions of multiple VMs over GPU. The hypervisor may be involved to a minimal extent.

3.2 GPU resource provisioning techniques in cloud

The GPU provisioning to vGPU-enabled VM in cloud data centers is done at four different levels.

3.2.1 Video card allocation

The video card allocation technique is responsible for allocating a GPU to a vGPU in the physical server. GPUs are housed in video cards, and the selection of video cards to be allocated for the given vGPU is taken care of by the video card allocation technique. There are three allocation techniques suggested by Siavashi et al. in their work [1]. The simple allocation policy follows First Fit policy; Breadth-First policy sorts available video cards in ascending order of their GPUs loads and returns lightly loaded card. Depth First policy sorts the video card in descending order of GPUs load and returns the card which is just enough to satisfy the vGPU resource needs.

3.2.2 GPU allocation to vGPUs

The GPU allocation technique is responsible for allocating a GPU in a video card (with multi-GPUs) to a requested vGPU. The three allocation techniques First Fit, Breadth-First, and Depth-First, described before are also used for GPU allocation.

3.2.3 GPU-enabled VM placement

VM placement policy for the VMs with vGPU attached is responsible for allocating a physical server in a data center. The First fit VM placement policy is used in VMware Horizon [12], where all the hosts are iterated until VM in question is accepted by a physical host considering VMs resource requirements.

3.2.4 GPU provisioning

The GPU provisioning technique is responsible for a sharing policy of physical GPU among multiple vGPUs. Some of the most commonly used GPU provisioning schemes [11] are listed below.

1. **Space shared:** The second vGPU is allocated only once the first vGPU is completed.
2. **Time shared:** The vGPUs share a physical GPU until co-executing vGPU does not exceed the total MIPS of a given GPU.
3. **FCFS:** First-come, first-served policy allocates vGPUs in the order that they arrive.
4. **Round-robin:** Round-robin is similar to FCFS but assigns a fixed time to vGPU. This policy is also called a fair share.
5. **Priority based:** Priority-based provisioning assigns a priority to every vGPU, and the provisioning logic executes vGPUs in the order of their priority.
6. **Fair queuing:** Fair queuing assigns start tag to every vGPU and schedule them for execution in increasing order of start tags. The accumulated usage time of a GPU is denoted by the start tag.
7. **Credit based:** The algorithm periodically distributes credits to vGPUs, and each vGPU consumes credits when it is executed on the CPU for exploiting the physical GPU. The policy selects a vGPU with a positive credit value.
8. **Affinity based:** This algorithm generates affinity scores for a vGPU to estimate the performance impact when it is allocated on a specific resource.
9. **SLA based:** SLA (Service Level Agreement) is an agreement between a cloud service provider and a user about the Quality of Service requirement and the price to be charged. SLA-based policy's objective is to meet the SLA requirement while allocating GPU resources.

3.2.5 Memory and PCIe bandwidth

The GPU device memory and PCIe bandwidth are two resources inside the GPU device that needs to be shared by co-running vGPUs. Each GPU can transfer data in two opposite directions simultaneously. PCIe bandwidth is provisioned on an equal share basis to all co-executing vGPUs. GPU Memory is one of the important bottleneck resources inside GPU, which may have an impact on the performance of co-executing vGPUs.

4 Problem definition

The proposed work in this paper investigates the problem of optimizing power consumption and improving the resource utilization of GPU resources in the cloud data centers by considering the physical machine composition, VM workload types, and GPU memory availability as parameters for VM placements on the physical hosts (PMs).

Two types of VM workloads are input to the proposed VM placement technique, VMs with associated GPU tasks and VMs without associated GPU task. The proposed technique considers the physical host composition to check if each active PM has the support of GPU processing or not. The technique also considers the dynamic GPU memory availability of all the available PMs. Based on incoming VM

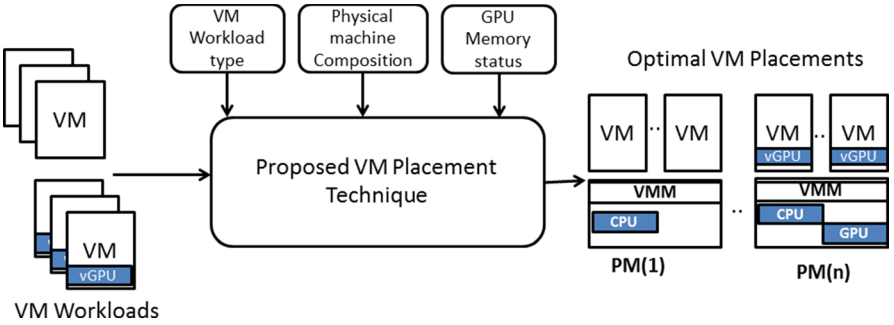


Fig. 4 System block diagram of proposed solution

workload type, an optimal mapping of a physical machine is found for placement using the proposed technique.

Our power consumption optimization problem is formulated as below,

$$\text{Ptotal} = \sum_{i=0}^{N} P(i) + (G(i).Pg(i)) \tag{1}$$

Ptotal denotes the total power consumption of cloud infrastructure, N denotes the number of physical machines at each data center. The $P(i) + (G(i).Pg(i))$ corresponds to total power consumption of ith machine. The $P(i)$ denotes the power consumption of PM except for GPU, the binary array $G(i)$ indicates if the machine has a physical GPU device or not. $Pg(i)$ denotes the power consumption of GPU if it exists on the ith physical machine. The block diagram of the proposed work in this paper is shown in Fig. 4.

5 First fit (FF) VM placement policy

The section describes the limitations of the first fit (FF) VM placement policy used in VMware Horizon [12] when employed in the GPU-enabled heterogeneous cloud data centers. The first fit (FF) VM placement policy searches a group of physical machines (PMs) sequentially to find first instance of a PM where the VM can be allocated and its resource demands can be met. The FF policy returns the Physical machine (PM) id in case the FF algorithm succeeds to find the suitable host. The FF technique works well in CPU only cloud data centers where the first physical host satisfying VMs resource demands is allocated to VM. However, it suffers from following drawbacks when used in the data centers supporting GPU-enabled instances.

5.1 Inefficient GPU utilization

The FF policy may place a non-GPU VM (with CPU only based processing) on physical host (PM) with GPU support if given PM satisfies the VM resource

demands. In such a case, the underlying GPU device remains underused. This situation may also cause starvation for some of the VMs with GPU tasks if other physical hosts (PMs) with GPU device are busy.

5.2 Non-optimal GPU-enabled VM placement

GPU device memory (GDDRAM) is an important bottleneck resource in GPU devices as it is shared between multiple vGPUs (virtual GPU instance) when placed on single physical GPU. Memory transfer operations between main memory (RAM) and GPU device memory in GPU task processing are performance bottlenecks and may cause higher response times if not avoided. The FF policy does not consider the GPU device memory availability across all physical hosts (PMs) before the placement of VMs. Inefficient GPU device memory usage can cause higher response time and power wastage. The proposed work presented in this paper attempts to overcome the drawbacks of the FF policy described in this section.

6 Proposed GPU-aware VM placement technique

The section proposes a metaheuristic-based technique for VM placements in the modern GPU-enabled cloud data centers. The proposed technique has the following sub-steps.

6.1 Classification of physical machines

The heterogeneous cloud data centers comprise of physical machines with varying computing power, composition, and energy efficiency. The cloud data centers supporting GPU computing host some of the physical machines with physical GPU devices attached. Such GPU fitted physical machines are enabled with a software (GPU driver) to process VMs with GPU tasks. It is critical to consider GPU task processing capability of physical machines for VM placement strategy to help reduce search space especially when VM placement requests contain a mixture of GPU and non-GPU VM workloads.

The characterization of physical machines as GPU-enabled and non-GPU machines and considering prioritizing PMs based on VM workload types improves the resource utilization of physical GPUs in the cloud data center. The proposed technique avoids placing a non-GPU VM on GPU attached physical machines unless no other physical hosts without GPU support are found suitable. It can be noted that First Fit (FF) policy does not classify physical machines based on GPU support and may result in inefficient VM placements such as placement of non-GPU VM on a GPU fitted physical machine. Also, the FF VM placement policy has to deal with a large search space for VM placements in heterogeneous data centers.

Algorithm 1: GPU based Physical machine classification

```
Input  : pmList
Output: GpuPmList, NonGpuPmList
Classify PMs based on the physical GPU device support
GpuPmList = new List();
NonGpuPmList = new List();
foreach pm ∈ pmList do
    if pm.GpudeviceQuery() == true then
        GpuPmList.add(pm);
    end
    if pm.GpudeviceQuery() == false then
        NonGpuPmList.add(pm);
    end
end
return;
```

The input to the algorithm 1 is the list of active heterogeneous physical machines available in the data centers containing mixture of GPU and non-GPU PMs. The algorithm 1 checks each physical machine for the GPU support through software layer API such as GpudeviceQuery(). The API queries the existence of GPU driver and GPU device details. If GPU support exists, the corresponding physical host is added to the list of GPU supported physical machines. If GPU device is absent, the physical host is added to the list of non-GPU physical machines. The classification of physical machines as GPU supported and non-GPU is done to reduce the search space for VM placements and improve the GPU device utilization in the data centers.

6.2 Prioritization of PMs based on GPU memory availability

The objective of the proposed technique is to improve the resource utilization of GPU device in the physical hosts of data centers. It can be noted from the GPU architecture block diagram in Fig. 1 that GPU device memory (GDDRAM) is one of the critical and bottleneck resources for the processing of multiple virtualized GPUs (vGPU) on single physical GPU device. The efficient VM allocation strategy based on GPU device memory availability status can optimize the VM placement process by accommodating more vGPUs on fewer physical hosts. The optimal utilization of GPU device memory is critical to achieving power efficiency and resource utilization in the data centers.

Algorithm 2: Sort PMs based on GPU Memory availability

```
   Input  : GpuPmList
   Output: GpuPmList
 1 Sort Physical machines based on GPU Device Memory in ascending order
 2 GpuAvailableMemArray = new List();
 3 foreach pm ∈ GpuPmList do
 4     minGpuMem = MAX_GPU_MEM_VALUE;
 5     foreach vc ∈ pm.getVideocards() do
 6         foreach gpu ∈ vc.getGpuList() do
 7             Integer AvailMem = gpu.getAvailableMemory();
 8             if AvailMem <minGpuMem then
 9                 minGpuMem = AvailMem;
10             end
11         end
12     end
13     GpuAvailableMemArray.add(minGpuMem);
14 end
15 SortAscendingOrder(GpuPmList, GpuAvailableMemArray);
16 return GpuPmList;
17
```

The proposed technique enables VM placement policy to consider GDDRAM availability of GPU devices attached to physical hosts for the VM placement process. The input to the algorithm 2 is the list of GPU supported physical machines. The algorithm 2 queries the GPU device memory availability of the physical hosts by iterating through all video cards present in each host and all GPU devices present in each video card. The algorithm prepares a list containing a key-value pair of each physical host and the smallest available GPU device memory in the corresponding host at that point in time. The list containing the pairs (PM id, GDDRAM value) is sorted in ascending order of GDDRAM availability by the algorithm and is returned as the output. The sorted physical machine list is utilized by algorithm 3 for searching a physical host for incoming VM placement requests.

6.3 GPU-aware best fit increasing VM placement technique

The proposed VM placement policy called GPU-aware Best Fit Increasing VM placement technique for heterogeneous data centers supporting GPU enable computing is described in this section. The proposed technique considers the heterogeneity in physical hosts based on GPU support and the dynamic GPU device memory availability in each physical host for VM placement decisions.

The input to the algorithm 3 is a VM placement request. The VM workload to be placed can be of two types. It can be a VM without any associated GPU tasks (VMcpu) or it can be a VM with associated GPU tasks (VMgpu). The result of the classification of physical machines based on GPU support done by algorithm 1 is

also input to the proposed algorithm 3 as the list of GPU PM List and non-GPU PM list.

The algorithm 3 checks the incoming VM if it has associated vGPU to be processed on a GPU device. If incoming VM contains associated vGPU, the algorithm sorts the GPU PM list in ascending order based on GDDRAM availability at that point in time as described in the algorithm 2. The VM is checked for placement suitability against each physical host fitted with GPU in ascending order of their dynamic GPU device memory availability using the best fit increasing search scheme. If incoming VM does not possess any vGPU processing request, then the VM is checked for placement suitability against non-GPU physical machines at first. If no non-GPU physical machines are found suitable, then VM is checked for placement suitability against GPU fitted machines. If VM is found a suitable physical host, the algorithm assigns success return code else a failure status to place VM is returned.

It can be noted that in the proposed algorithm the non-GPU physical machines are prioritized for VMs with no vGPU associated with them. This is done to avoid placement of non-GPU VM on a GPU fitted physical machines causing possible wastage of GPU resources in the data centers. The algorithm also ensures GPU resources are utilized optimally by using the best fit increasing (BFI) search scheme among GPU-enabled physical machines for VM placements involving vGPUs.

Table 2 Evaluation metrics used for experiments

Serial no.	Performance metric	Description
1	Energy consumption	The metric denotes the total power consumed by all physical machines and GPUs in the cloud data centers. Lower power consumption means higher power efficiency and cost savings. The energy consumed is measured in Kilo Joules
2	The total number of host searches for VM placements	The metric denotes the total number of PMs searched for requesting VM placements. The metric denotes how quickly the algorithm finds a suitable PM. A lower number of host searches to arrive at suitable PM are preferred provided the VM-PM mapping is apt in terms of power consumption and resource utilization
3	Total time taken by the VM placement algorithm	The metric denotes the total running time of the VM placement algorithm. The lower processing time to meet desired VM placement goals imply a reduction in CPU usage and improvement in response times for the end-users. The total time taken is represented in milliseconds

Algorithm 3: GPU aware Best Fit Increasing VM Placement

```
   Input  : Vm, GpuPmList, NonGpuPmList
   Output: VmAllocationResult
 1 Gpu workload aware Host Search for VM placement
 2 VMAllocationResult = false;
   /* Case 1 - VM with GPU tasks                                    */
 3 if Vm.vGpu() ≠ NULL then
 4     MemoryAwareSortGpuHosts(GpuPmList);
 5     foreach pm ∈ GpuPmList do
 6         if pm.isSuitabe(Vm) then
 7             pm.Allocate(Vm);
 8             VMAllocationResult = true;
 9             return VMAllocationResult;
10         end
11     end
12 end
   /* Case 2 - VM with No GPU tasks                                 */
13 if Vm.vGpu() == NULL then
14     //Search Non-Gpu Host for Non-Gpu VM
15     foreach pm ∈ NonGpuPmList do
16         if pm.isSuitabe(Vm) then
17             pm.Allocate(Vm);
18             VMAllocationResult = true;
19             return VMAllocationResult;
20         end
21     end
22     //Search Gpu Host for Non-Gpu VM
23     foreach pm ∈ GpuPmList do
24         if pm.isSuitabe(Vm) then
25             pm.Allocate(Vm);
26             VMAllocationResult = true;
27             return VMAllocationResult;
28         end
29     end
30     end
31     return VMAllocationResult;
32
```

Table 3 Experimental configurations used for evaluation

Exp. no	Total number of PMs	Number of GPU fitted PMs	Number of non-GPU PMs	Total number of VMs	Number of VMs with GPU tasks	Number of VMs without GPU tasks
1	8	4	4	56	28	28
2	16	8	8	112	56	56
3	32	16	16	224	112	112
4	1024	512	512	7168	3584	3584
5	1024	256	768	7168	1792	5376

Table 4 Physical machine configurations used for evaluation

PM type	Model name	Frequency (GHz)	Cores	RAM (GBs)	GPU card	Power (W)
GPU-fitted PM	IBM Dual Intel Xeon E5-2620 V3	2.4	12	64	GRID K1	330
Non-GPU PM	IBM Dual Intel Xeon E5-2620 V3	2.4	12	64	Not available	330

Table 5 Specification of physical GRID graphics card

Video card model	Composition	
GRID K1	Number of GPUs	4
	Frequency	850 MHz
	GPU SMs	1
	GDDRAM	4 ×4 GBs
	TDP	130 W

Table 6 Specification of VM workload types used in experiment

Serial no	VM type	MIPS	RAM (GB)	VM size (GB)	# Cores	Network BW (Mbps)	vGPU type
1	VM (gpu)	38,400	2	10	1	100	GRID K180Q
2							GRID K160Q
3							GRID K140Q
4	VM (non-gpu)	38,400	2	10	1	100	Not available

Table 7 Specification of vGPUs used in experiment

S.no	vGPU type	GDDRAM (MB)	vGPUs/GPU	vGPUs/card
1	GRID K180Q	4096	1	4
2	GRID K160Q	2048	2	8
3	GRID K140Q	1024	4	16

7 Performance evaluation

The evaluation of the GPU-aware VM placement technique is carried out using GPUCloudsim [1] simulation toolkit. The evaluation results from the proposed technique are compared against a First Fit (FF) policy for VM placements used in the VMware Horizon [12].

7.1 Evaluation metrics

The various evaluation metrics used for our experiments are described in Table 2. The goal of our evaluation is to testify our proposed VM placement technique for total energy consumption, search space minimization and processing time efficiency.

7.2 Experimental setup

The GPUCloudsim [1] simulation framework is employed for the evaluation of our proposed GPU-aware VM placement technique. GPUCloudsim is a simulation framework based on the well-known CloudSim [4] architecture. GPUCloudsim helps researchers in conducting experimental evaluations of the proposed algorithms for the GPU-enabled cloud data centers. The GPUCloudsim framework help to evaluate the effectiveness of various GPU virtualization schemes, GPU scheduling, and provisioning policies before being deployed on the real-world GPU-enabled cloud data centers. The GPUCloudSim simulation tool offers a flexible, extensible and configurable open-source simulation environment for the researchers to verify their cloud-based solutions.

All the experiments are conducted on an HP Probook laptop (with Core i7-8650 CPU with 16 GB RAM) running Windows 10 operating system. The experiments are designed to emulate GPU-enabled data centers of varying size and compositions by having a mixture of variable number of GPU fitted servers (PMs) and CPU only

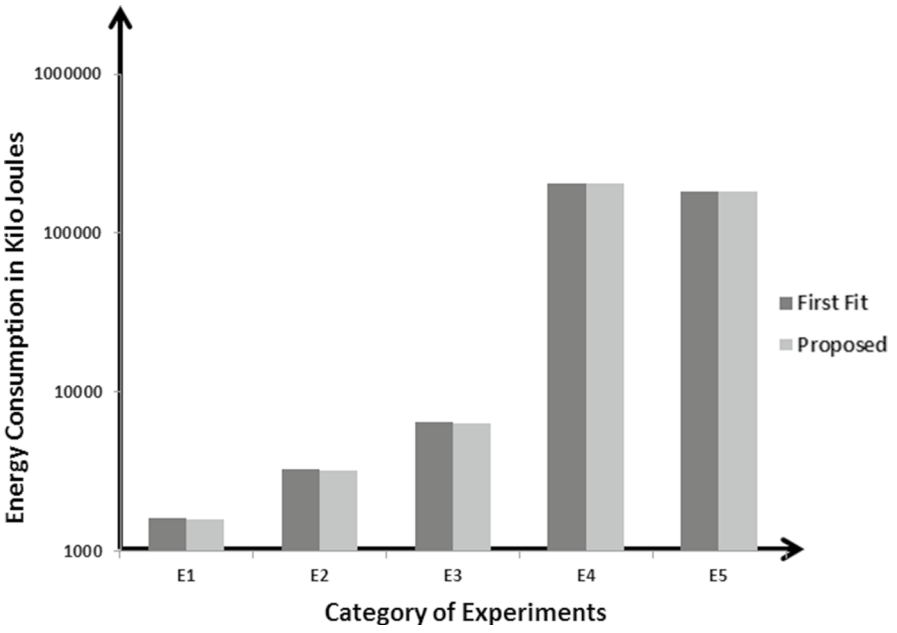


Fig. 5 Comparison of energy consumption

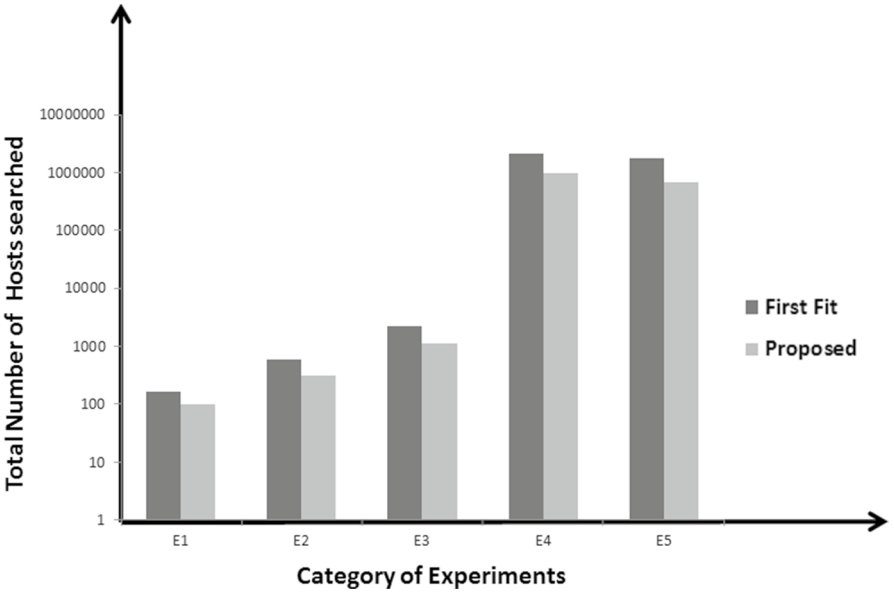


Fig. 6 Comparison of total hosts searched for VM placements

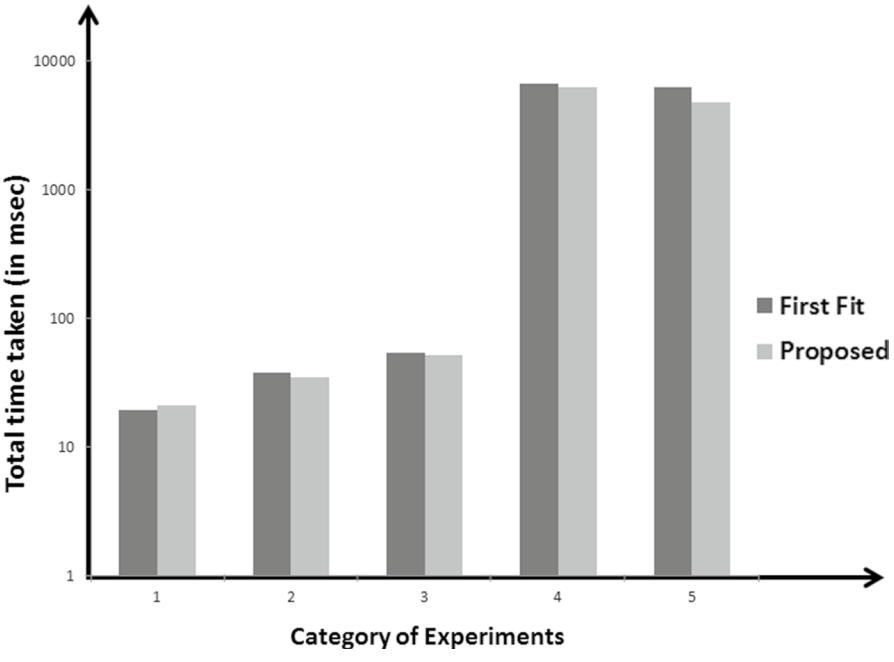


Fig. 7 Comparison of total time taken by VM placement technique

Table 8 Evaluation results for performance metrics

Experiment number	Energy consumption (in KJ)		Total Number of hosts searched for VM placements		Total time taken by VM placement (in msec)	
	First fit based	Proposed technique	First fit based	Proposed technique	First fit based	Proposed technique
1	1603.40	1579.22	160	100	19.19	20.98
2	3208.76	3160.41	576	310	37.82	34.83
3	6416.55	6319.85	2178	1100	53.28	51.80
4	205,361.14	202,266.63	2,118,214	983,028	6558.75	6228.43
5	181,496.08	179,948.83	1,725,000	676,812	6154.85	4744.00

servers. The input VM workload configurations are designed to contain a mixture of VMs with GPU tasks (VMgpu) and VMs without GPU tasks (VMnon-gpu). Each VMgpu contains one of the three different types of vGPUs (as shown in Table 7) having varying resource demands for memory and GPU for evaluation. The experiments E1-E5 containing the varying composition of physical machines and virtual machine compositions are described in Table 3. The goal of the evaluation is to testify the proposed VM placement technique against a mixture of VM requests which vary in terms of processing types (such as CPU-based tasks, CPU tasks along with GPU tasks) and diverse GPU device memory (GDDRAM) demands.

To simulate GPUs, we study the use of NVIDIA GRID technology for GPU virtualization. GRID technology enables hardware virtualization intended for cloud environments. GRID technology allows multiple vGPUs to share one physical GPU at the same time. The physical machine types (GPU-fitted and non-GPU) and their compositions used for experiments are described in Table 4. The specification of physical GRID GPU simulated with GPU fitted physical machines is given in Table 5. It is assumed that cloud data center use VMWare Horizon [12] to support GPU-enabled processing. In each physical server (PM), the PCIe bandwidth is shared equally among available GPUs in the video card. The VM workload configurations used in the experiments are tabulated in Table 6. The configuration of vGPU types associated with VMgpu VM types are described in Table 7. The listed vGPU types in Table 7 can be allocated on a PM with GRID K1 video card. Each VM(gpu) contains one of vGPU type mentioned in Table 7. The vGPU types are distributed over total VMs(gpu) considered for each experiment such that requested vGPU resources do not exceed available physical GPU capacity.

7.3 Evaluation results

The evaluation results for the experimental configurations tabulated in Table 3 are presented in Table 8. The results presented in Table 8 contain the measurements for three performance metrics over five experiments. It can be noted from Fig. 5 that the energy consumption for the proposed technique is lower than the First fit technique for all the experimental configurations. The energy consumption is reduced by 0.8–1.5% with the proposed GPU-aware VM placement technique. Figure 6 shows the total number of physical hosts (PMs) checked for the suitability of VM placements by the First fit and the proposed technique.

It can be observed from Fig. 6 that the proposed technique reduced the target physical machine search space by 37–60% for VM placements. It is evident from Fig. 6 that the proposed technique for VM placement can minimize the search space by a significant amount for any combination of non-GPU physical hosts and GPU fitted physical hosts. Figure 7 represents the total time taken by the First fit technique and proposed GPU-aware VM placement technique for completing all the VM placements. It can be observed from Fig. 7 that for experimental configurations E2 to E5, the total time taken (makespan) by the proposed technique is reduced by 2–23%. However, for experimental configuration E1, the proposed technique is found to be higher by 10%. The higher makespan for E1 can be attributed to higher number of

GPU specific computations given the total size of the experiment. It can be attested from the evaluation results that the proposed GPU-aware VM placement technique can reduce total energy consumption and reduce the target physical machine search space for VM placements by significant margins. The average makespan of the proposed technique is found to be reduced by over 13% for the experimental configurations considered.

8 Current challenges with GPU computing in the cloud

Virtualization is a crucial technology for the efficient utilization of server resources in cloud data centers. The virtualization solutions for CPU, memory and network have attained sufficient maturity to be used in data centers for the benefit of both cloud providers and users. However, the same conventional technologies in virtualization do not apply for the GPU virtualization because of inherent differences in architecture, programming models, and vendor-specific driver software.

Our work investigates various issues with current frameworks and technologies employed for GPU processing that limits VMs ability to exploit the real power of GPUs under the virtualization layer. The proposed work also examines the several gaps/issues in efficient GPU utilization by current resource provisioning techniques.

8.1 Resource management perspective

The section discusses various system-level issues that prevent efficient resource management of GPUs and GPU attached servers inside cloud data centers.

8.1.1 GPU-enabled VM migration

VM migration process is re-placement of VM from source physical host to a destination physical host in data centers. The VM migrations are usually done for performance optimization, to avoid resource contentions, and to perform server consolidation during non-peak load situations. Though there are many proven algorithms [5] existing for the VM migration process, the process gets complicated when GPU-enabled VM is to be migrated. The vGPU attached to the VM will have its process state and data inside GPU memory. The VM migration process has to wait till the application finishes its GPU tasks or the ongoing GPU tasks need to be aborted in the source machine and restarted in the destination host machine. The extra computation or the extra delay caused by vGPU computation causes inefficiency in the VM migration process.

When GPUs in cloud servers are virtualized using hardware-assisted virtualization technology like NVIDIA Grid [19], such a virtualization technology bypasses the core virtualization layer in the server for creating and managing virtual images of GPU. When vGPU has to be live migrated from such hardware-assisted virtualized GPU, retrieving GPU task states and restoring it on remote GPU will be an

intricate process. There is a need to establish new mechanisms to live migrate hardware-assisted virtualized GPU images from one GPU to remote GPU efficiently.

8.1.2 Power modeling of GPUs

Various vendors manufacture the GPUs, and the architecture of GPUs are inherently different from one another. Because of their composition, the power consumption and performance characteristics will be varying from one another. Unlike CPUs, the power consumption and performance benchmark results [24] for GPUs are not available yet. The power-aware resource provisioning policies for GPUs have to rely on only the mathematical model for power consumption estimation. The mathematical equation [1] for power consumption analysis is given by Eq. (2).

$$P(f, U) = a3.f.U + a2.f + a1.U + a0 \tag{2}$$

It is concluded from a set of experiments where power is measured by running a server under different GPU utilization and frequencies by applying an curve fitting method that there exists a linear correlation between the energy consumption with frequency and utilization of SMs in GPU. In Eq. (2), the frequency f and utilization U determine the power consumption approximation where *ai*s are curve fitting constants determined by a specific curve fitting method [8]. Because the power consumption approximation is based on a mathematical model, the inherent physical composition of GPUs contributing to the power consumption factor is not considered. The factor may impact the efficiency of the resource provisioning algorithms.

8.1.3 Power saving strategies involving GPUs

The power-saving in the data center is carried out by shutting down some of the servers with lower CPU utilization during non-peak hours of data center operations. However, when a physical server is equipped with one or more GPUs, the power saving strategy that selects a physical host for power-off has to consider additional parameters such as GPU utilization and state of GPU tasks to make host power-off decisions. Also, the performance and power characteristics of both CPU and GPU can be considered for the selection of an underutilized host for power-off. The underutilized servers with relatively less power efficient GPUs can be prioritized for power-off to maximize the utilization of power-efficient GPUs.

8.1.4 Data center context-adaptive GPU allocation policies

The video card allocations, GPU allocation policies have to adapt to changing load conditions in data centers. The allocation techniques employed have to be aware of datacenter load (peak or non-peak conditions) state [16] to make optimal decisions for allocations. For example, during peak hours, the allocation policy can follow the breadth-first scheme or first fit technique for allocations. During non-peak hours, the allocation technique can employ a depth-first policy for the video card or GPU allocations. Such adaptive schemes can make GPU allocation models more responsive

to the objective of power-saving during non-peak hours and the objective of high performance during peak hours.

8.1.5 GPU memory pollution with fair share policy for GPU provisioning

The fair share policy for GPU provisioning allocates a time slice of physical GPU to many vGPUs in round-robin fashion. GPU provisioning policies like fair share will have to make many hosts to device and device to host data transactions. If the data size is not small, the vGPUs switching makes the processing very inefficient because of multiple main memory and GPU device memory transactions. Memory transfers are considered as major bottlenecks to achieve high throughput, and GPU device memory size limits the level of multitasking on GPU if data associated with GPU tasks is large. It is vital to consider data transactions between host and GPU for GPU provisioning.

8.2 Perspective of GPU-enabled applications

The section describes various application side challenges that prevent exploiting the true power of GPUs by accessing them from the top of the virtualization layer.

8.2.1 Target GPU generations

The GPUs available in the market possess different computing capabilities and support a varying degree of features because of their generation and type. If applications are ignorant of the target GPU generation or version, the application design may not truly exploit the physical GPU's power. For example, the device memory sizes, shared memory size, and a number of SIMD cores will vary between generations of GPU, and such details will impact the performance and QoS guarantee for users in real-time applications.

8.2.2 Heterogeneity in GPUs and multiple frameworks

Multiple vendors manufacture data center-class GPUs in the market, and they possess different hardware extensions to support various features such as deep learning, AI solutions. Such heterogeneity in GPUs makes VM re-placement a complex process. If VMs with GPU Tasks include demands for such additional features supported inside GPU, then such additional constraints need to be considered for VM placement.

The GPGPU programs use different frameworks (CUDA, OpenCL, Vulcan, etc.) for accessing and computing on GPUs. Application using the CUDA framework for their GPU Tasks inside VMs should be allocated with NVIDIA GPUs inside data centers. The resource management module for GPU provisioning needs to consider such hardware and software related constraints for making VM placements.

8.2.3 Security aspects

Some GPU frameworks delay the GPU specific source code compilations and GPU executable binary generation till runtime if target GPU device makes, version or generation is not known beforehand. Such VMs with delayed compilation process should be placed in secured environments to avoid cross-VM attacks as application logic may be leaked and used with malicious intent. The GPU allocation and provisioning policy have to consider such constraints for VM placements.

Some VM applications [11] may pose a denial of service attack for GPU by submitting a massive number of GPU tasks to underlying GPU devices and deny GPU resources for other co-allocated VMs. There is a need for new control mechanisms in the GPU virtualization layer to detect and control such VMs from overusing the GPU device.

The system-level and application-specific issues with GPU computing in virtualized environments discussed in this section can be considered for designing an efficient GPU resource management framework in cloud data centers to improve performance and efficient resource utilization.

9 Conclusion

In recent years, GPU computing in cloud data centers is gaining momentum swiftly because of the massive parallel computing demands from applications like deep learning, virtual desktop infrastructure applications, etc. Though CPU virtualization techniques are matured enough, GPU virtualization and resource management is still an area of active research. The VM placement techniques involving GPU allocation and provisioning needs further consideration for additional parameters and constraints. The paper proposes a GPU-aware VM placement optimization technique that considers the VM workload type and GDDRAM availability for VM placement decisions in heterogeneous data centers supporting GPU computing. It is evident from the evaluation results that the proposed technique can perform better than the First fit VM placement technique in a GPU-enabled virtualized environments. The paper also summarized the underlying architecture of GPU, current GPU virtualization software, and hardware infrastructure available in cloud data centers. The paper also discussed various challenges investigated from GPU resource management and application perspectives to motivate further research in the related domain. The proposed work in this paper does not consider GPU-enabled VM migrations, which can be considered as future scope of the proposed work. Our further research is focused on solving some of the resource management issues discussed in this paper to improve GPU-enabled VM placement, GPU resource provisioning, and optimization algorithm.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Ahmad S, Mahmoud M (2019) GPUCloudSim: an extension of CloudSim for modeling and simulation of GPUs in cloud data centers. J Supercomput 75:2535–2561
2. Bakhoda A, Yuan GL, Fung WW, Wong H, Aamodt TM (2009) Analyzing CUDA workloads using a detailed GPU simulator. In: International symposium on performance analysis of systems and software ISPASS. IEEE, pp 163–174
3. Blythe D (2006) The Direct3D 10 system. In: ACM transactions on graphics (TOG), vol 25. ACM, pp 724–734
4. Calheiros R, Ranjan R, Beloglazov A, De Rose C, Buyya R (2011) CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. Softw Pract Exp 41:23–50
5. Choudhary A, Govil MC, Singh G, Awasthi LK, PilliEmail ES, Kapil D (2017) A critical survey of live virtual machine migration techniques. J Cloud Comput Adv Syst Appl 6:23
6. Farooqui N, Barik R, Lewis BT, Shpeisman T, Schwan K (2016) Affinity were work-stealing for integrated CPU-GPU processors. In: Proceedings of the 21st ACM SIGPLAN symposium on principles and practice of parallel programming. ACM
7. Gong X, Ubal R, Kaeli D (2017) Multi2Sim Kepler: a detailed architectural GPU simulator. In: International symposium on performance analysis of systems and software (ISPASS). IEEE, pp 153–154
8. Guan H, Yao J, Qi Z, Wang R (2015) Energy-efficient SLA guarantees for virtualized GPU in cloud gaming. IEEE Trans Parallel Distrib Syst 26(9):2434–2443
9. Gupta V, Gavrilovska A, Schwan K, Kharche H, Tolia N, Talwar V, Ranganathan P (2009) GViM: GPU-accelerated virtual machines. In: Proceedings of the 3rd ACM workshop on system-level virtualization for high performance computing. ACM, pp 17–24
10. Gupta V, Schwan K, Tolia N, Talwar V, Ranganathan P (2011) Pegasus: coordinated scheduling for virtualized accelerator-based systems. In: USENIX Annual Technical Conference
11. Hong C-H, Ivor S, Nikolopoulos DS (2017) GPU virtualization and scheduling methods: a comprehensive survey. ACM Comput Surv 50
12. Horizon–virtual desktop infrastructure. https://www.vmware.com/products/horizon.html. Accessed on July 2019
13. Huang Q, Huang Z, Werstein P, Purvis M (2008) GPU as a general purpose computing resource. In: 2008 Ninth International Conference on Parallel and Distributed Computing, Applications and Technologies, Dunedin, New Zealand, pp 151–158
14. Khronos Group, Vulkan, Version 1.1. https://www.khronos.org/vulkan/. Accessed on July 2019
15. Khronos OpenCL Working Group and others (2008) The OpenCL specification. Version 1
16. Kulkarni AK, Annappa B (2019) Context-aware VM placement optimization technique for heterogeneous IaaS cloud. In: IEEE access, pp 1–3
17. Mell PM, Grance T (2011) SP 800-145. The NIST definition of cloud computing. Technical report. National Institute of Standards and Technology, Gaithersburg, MD, USA
18. Menychtas K, Shen K, Scott ML (2014) Disengaged scheduling for fair, protected access to fast computational accelerators. In: ACM SIGPLAN notices, vol 49. pp 301–316
19. NVIDIA GRID Technology. https://www.nvidia.com/en-us/data-center/virtual-gpu-technology/. Accessed on July-2019
20. NVidia, Deep Learning. https://developer.nvidia.com/deep-learning. Accessed on March, 2021
21. Ogata K (2007) CUDA-Compute unified device architecture programming guide, NVIDIA
22. Sengupta D, Belapure R, Schwan K (2013) Multi-tenancy on GPGPU-based servers. In: Proceedings of the 7th international workshop on virtualization technologies in distributed computing. ACM, pp 3–10
23. Suzuki Y, Kato S, Yamada H, Kono K (2014) GPUvm: why not virtualizing GPUs at the hypervisor?. In: USENIX Annual Technical Conference (USENIX ATC 14). pp 109–120
24. The SPECpower benchmark. http://www.spec.org/power_ssj2008/. Accessed on July, 2019

25. Vmware (2019). Virtualization. https://www.vmware.com/in/solutions/virtualization.html. Accessed on Sep, 2019
26. Woo M, Neider J, Davis T, Shreiner D (1999) OpenGL programming guide: the official guide to learning OpenGL, version 1.2. Addison-Wesley Longman Publishing Co., Inc
27. Zhang C, Yao J, Qi Z, Miao Yu, Guan H (2014) vgasa: adaptive scheduling algorithm of virtualized gpu resource in cloud gaming. IEEE Trans Parallel Distrib Syst 25(11):3036–3045

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.